

Q1. (a)

No	Time Complexity	Big- θ notation
(i)	$12n \log_2 n^{20} + 1570n$	$\Theta(n \log n)$
(ii)	$(40n)^{99} + n^{2000}(\log_2 n)^{100} + 0.01(6^n) + 2(2^n)$	$\Theta(6^n)$
(iii)	$(n-1)! + n^{80} + 40n^{40} + n!$	$\Theta(n!)$
(iv)	$369n^2 + 246n \log_2 n + 999511911$	$\Theta(n^2)$
(v)	$9 \log_2 n^{1001}$	$\Theta(\log n)$
(vi)	$39n \log_2 n + 2n \log_2 \log_2 n + 13579$	$\Theta(n \log n)$

(b) $\exists c=100, n_0=1 \bullet \forall n > n_0 \bullet$

$$n > 1$$

$$n^3 < n^4$$

$$100n^3 < 100n^4$$

$$100n^3 < cn^4.$$

(c) Need to prove $[\exists c, n_0 \bullet \forall n > n_0 \bullet 100n^3 > cn^4]$ is wrong.
 Equivalent to prove for $[\forall c, n_0 \bullet \exists n > n_0 \bullet 100n^3 \leq cn^4]$
 For any c, n_0 , take $n = \max(n_0 + 1, \lceil 100/c \rceil) > n_0$, we have

$$n \geq 100/c$$

$$100 \leq cn$$

$$100n^3 \leq cn^4.$$

(d)

$$2^{2n}$$

$$= 4^n$$

$$= 4 * 4 * 4 * 4 * 4 * 4 * \dots * 4 \quad \text{for } n \text{ factors}$$

$$< 1 * 2 * 3 * 4 * 5 * 6 * 7 * \dots * n \quad 4^3 \text{ are settled by 16 and 64}$$

$$= n!$$

$$= O(n!)$$

OR

$$< 4 * 4 * 4 * 4 * 5 * 6 * 7 * \dots * n$$

$$< 4 * 4 * 4 * (1 * 2 * 3 * 4 * 5 * 6 * 7 * \dots * n)$$

$$= 64 n!$$

$$= O(n!)$$

Q2. (a)

Quicksort	Time Complexity
Best case	$O(n \log n)$
Average case	$O(n \log n)$
Worst case	$O(n^2)$

(b)

Quicksort	Recurrence for time complexity
Best case	$T(n) = n + \min(T(1)+T(n-1), T(2)+T(n-2), \dots, T(n-1)+T(1))$
Average case	$T(n) = n + (1/(n-1))(T(1)+T(n-1), T(2)+T(n-2), \dots, T(n-1)+T(1))$
Worst case	$T(n) = n + \max(T(1)+T(n-1), T(2)+T(n-2), \dots, T(n-1)+T(1))$

(c) Claim: $T(n) < c n \log n$, $c \geq 10$, $n > 10$

$$\begin{aligned} T(n) &= T(n/10) + T(9n/10) + n \\ &< c(n/10)\log(n/10) + c(9n/10)\log(9n/10) + n \\ &= cn \log n - c(n/10)\log 10 - c(9n/10)\log(10/9) + n \\ &< cn \log n - c(n/10) + n \\ &= cn \log n, \text{ it is why } c \geq 10 \text{ is needed.} \end{aligned}$$

(d) $T(n) = T(\lceil n/10 \rceil) + T(\lfloor 9n/10 \rfloor) + n = O(n \log n)$

(e) $O(n \log n)$

Q3. (a)

(i)

```
Algorithm sum(i,m)
  if i=0 or m=0 then return 0;
  if  $v_i > m$  then
    return sum(i-1,m);
  else
    return max( $v_i + \text{sum}(i-1, m - v_i)$ , sum(i-1,m));
```

(ii)

```
// input is  $v_1, v_2, \dots, v_n$ , and M
// sum(0..n, 0..M) is an array.

for m=0 to M
  sum(0,m)=0;
for i=0 to n
  sum(i,0)=0;
for i=1 to n
  for m=1 to M
    if  $v_i > m$  then
      sum(i,m)=sum(i-1,m);
    else
      sum(i,m)=max( $v_i + \text{sum}(i-1, m - v_i)$ , sum(i-1,m));
```

- (b) Suppose the input is $a_1, a_2, \dots, a_n$. Let m_{ij} be the optimal cost of full parenthesization from the i th element to the j th one. Let s_{ij} be the sum of elements from the i th element to the j th one.

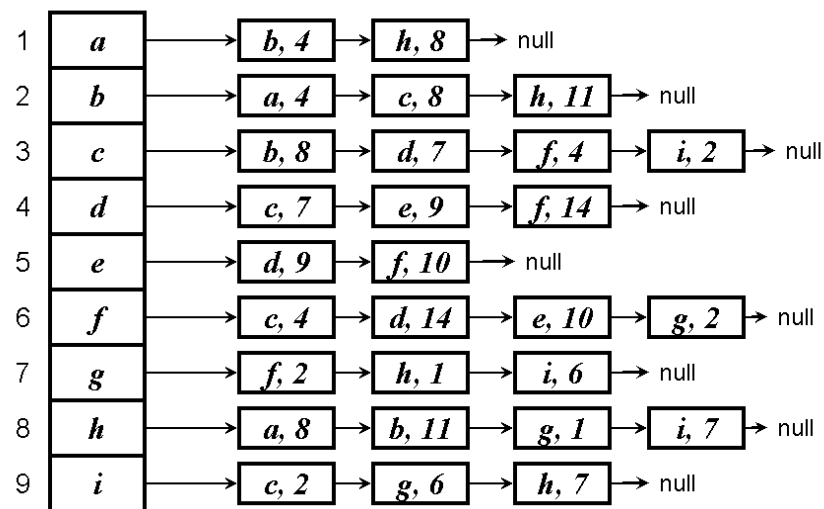
For all i , $m_{ii}=0$, $s_{ii}=a_i$.

For $i < j$,

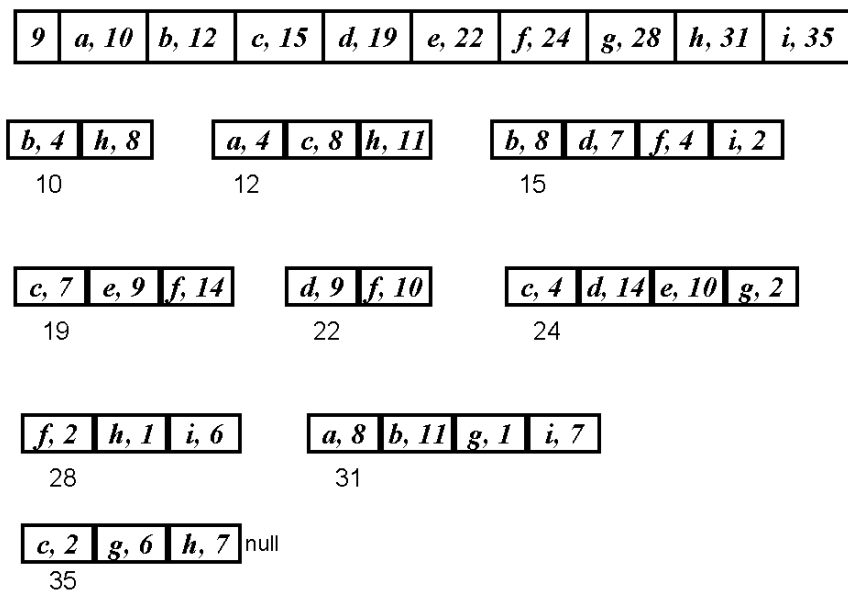
$$s_{ij} = s_{i, j-1} + a_j \text{ and } m_{ij} = \min_{k=i \text{ to } j-1} \{m_{ik} + m_{k+1, j} + s_{ij}\}.$$

Q4. (a)

(i)



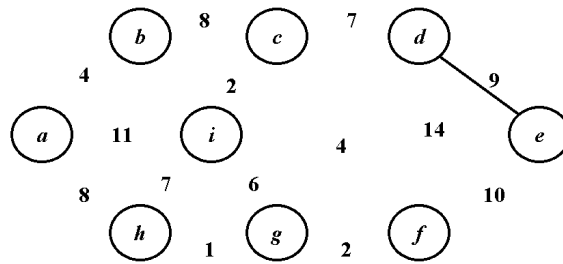
(ii)



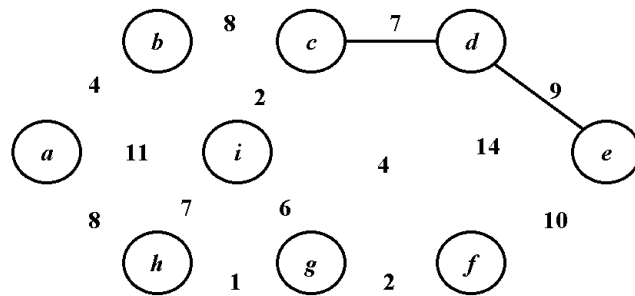
The above segments are actually a long sequence of cells.

The first number is #node. Then following by 9 cells. Each cell has an node id, and a pointer pointing to the beginning position of the adjacent list.

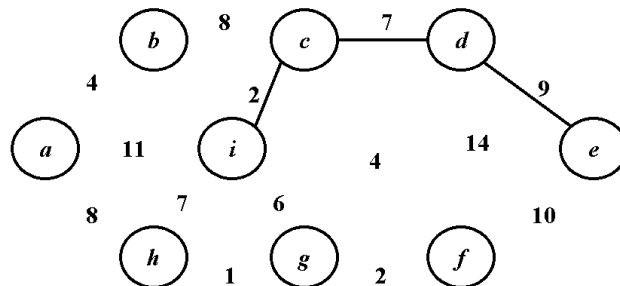
(iii) step 1:



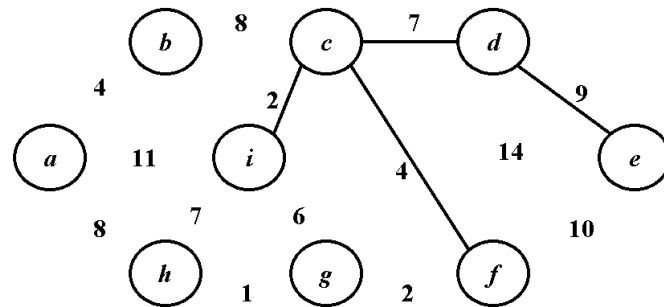
step 2:



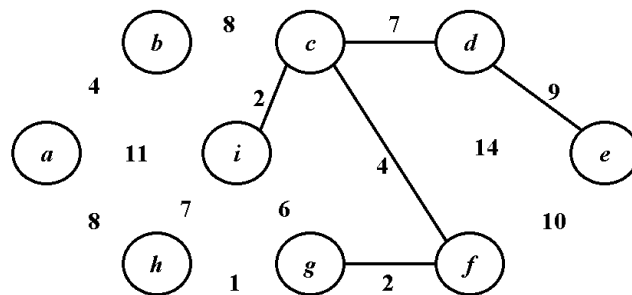
step 3:



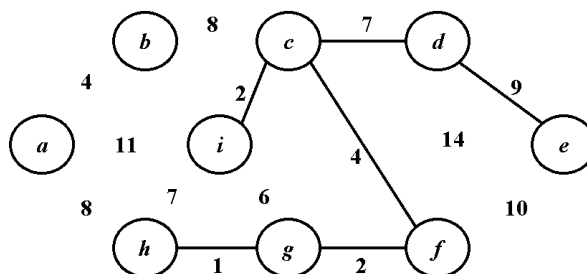
step 4:



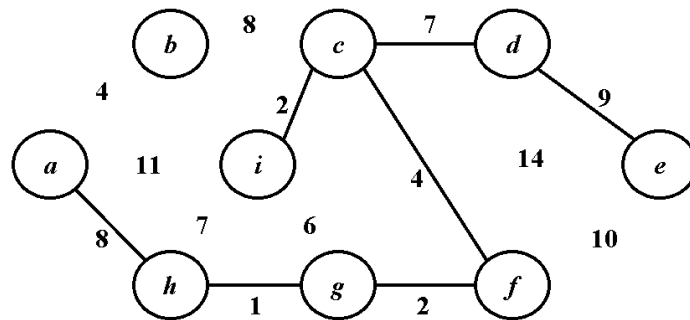
step 5:



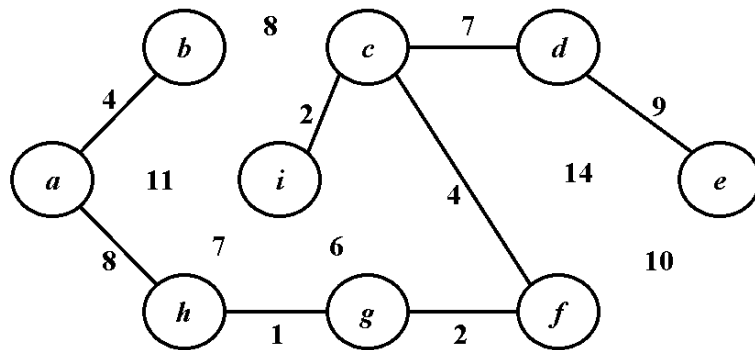
step 6:



step 7:



Step 8:



(b) All edges have unique weights.

Q5. (a) input: a binary tree T
output: the height of T

```

Algorithm height( $T$ )
  if  $T = \emptyset$  then return 0
   $hL = \text{height}(T \rightarrow \text{left});$ 
   $hR = \text{height}(T \rightarrow \text{right});$ 
  return  $\max(hL, hR) + 1;$ 

```

- (b) input: a binary tree T
output: the number of nodes of T

```

Algorithm num_node(T)
    if T =  $\emptyset$  then return 0
    nL = num_node(T->left);
    nR = num_node(T->right);
    return nL+nR+1;

```

- (c)

```

Algorithm test(n,h)
    m=1;
    for j=1 to h
        m=m*2;
    if n=m-1 then return true else return false;

```

- (d)

input: a binary tree T
output: true if T is full; else, false.

```

Algorithm full(T)
    h=height(T);
    n=num_node(T);
    return test(n,h);

```

- (e)

Proof:

IF any binary tree of height h has at most $2^{h-1} - 1$ nodes, where $1 \leq h < H$.

Consider a binary tree of height H.

By the definition of height, one of the left and right subtrees of the root must have height H-1. This subtree has at most $2^{H-2} - 1$ nodes by the given condition.

Let h' be the height of the other subtree, where $0 \leq h' \leq H-1$.

If $h' > 0$, then, by the given condition, #node inside is at most $2^{h'-1} - 1 \leq 2^{H-2} - 1$.
Summing up all nodes, including the root, total #node $\leq 2^{H-1} - 1$.

If $h' = 0$, by the definition of height, this subtree is empty. Summing up all nodes, including the root, total #node $\leq 2^{H-2} - 1 + 1 < 2^{H-1} - 1$.

Therefore, any binary tree of height H has at most $2^{H-1} - 1$ nodes for any cases.

- (f)

Need a base case of $h=1$.

Need two base cases of $h=0$ and $h=1$.

Counter example can be found.

any one of the above, or other reasonable ones.